Cours : comprendre climb_env_cfg.py

Nov 1, 2026 · @Bettcher

1. L'idée générale : une recette, pas un programme

`climb_env_cfg.py` ne simule rien et n'entraîne rien : ses fonctions renvoient une **description** de l'environnement, comme une recette. C'est `uv run train` qui lit la recette, puis mjlab qui construit tout (les milliers de robots sur le GPU, les managers, la boucle d'entraînement).

```
climb_env_cfg.py  ──(la recette)──▶  mjlab  ──(construit)──▶  6144 robots + règles + PPO
```

Tout le fichier repose sur un seul motif :

```
RewardTermCfg(func=mdp.limb_hooked, weight=5.0, params={"sockets": goal, "limb": LEFT_FOOT})
```

À chaque pas, mjlab appelle `func(env, **params)` et multiplie le résultat par `weight`. La fonction calcule la valeur pour tous les robots à la fois et renvoie un tableau d'un nombre par robot.

Toutes les classes dont le nom finit par `Cfg` (pour « configuration ») suivent ce schéma : ce sont des **fiches de réglages** (des dataclasses Python, comme une `struct` en C) qui contiennent une fonction, ses paramètres et parfois un réglage en plus (poids, bruit, mode). Toi, tu remplis des fiches ; mjlab les lit et construit les objets qui calculent.

2. Imports et constantes

Les imports se rangent en trois familles :

Famille	Exemples	Ce que c'est
Fonctions toutes faites de mjlab	`base_ang_vel`, `joint_pos_rel`, `time_out`, `action_rate_l2`	observations, récompenses et terminaisons classiques
Classes de configuration	`RewardTermCfg`, `SceneCfg`, `MujocoCfg`	les fiches qu'on remplit
Notre code	`climber_cfg`, `wall_spec_fn`, `mdp`, `Wall`, `Hold`	le robot, le mur, nos fonctions d'escalade

`mdp.quelquechose` désigne une fonction de **notre** fichier `mdp.py` : observations du critic (`ball_offsets`, `hooked_flags`), récompenses et terminaisons (`trunk_dropped`, `limbs_lost`).

Le temps. `TIMESTEP = 0.004` : la physique avance par pas de 4 ms. `DECIMATION = 5` : le robot décide tous les 5 pas de physique, soit 5 × 4 ms = **20 ms** par décision (50 par seconde, comme le vrai robot). Entre deux décisions, il garde la même consigne.

Les numéros des membres. Les 4 boules sont toujours rangées dans l'ordre de `LIMBS` : 0 main gauche, 1 main droite, 2 pied gauche, 3 pied droit. Les fonctions de `mdp.py` travaillent avec ces numéros :

```
LEFT_FOOT = LIMBS.index("left_foot")                                    # 2
OTHERS_THAN_LEFT_FOOT = tuple(i for i in range(4) if i != LEFT_FOOT)  # (0,
1, 3)
```

On les calcule au lieu d'écrire `2` et `(0, 1, 3)` en dur : c'est plus lisible, et si l'ordre de `LIMBS` changeait, les numéros suivraient au lieu de désigner silencieusement le mauvais membre.

3. La base commune : _base_cfg

`_base_cfg` contient tout ce qui est commun aux tâches. Le `_` au début du nom signale une fonction interne : les tâches l'appellent pour obtenir une base, puis y ajoutent leurs règles.

3.1 Les ingrédients

```
wall = Wall.load(WALL)
start_sockets = [s.tolist() for s in wall.start_sockets()]
robot = SceneEntityCfg("robot", joint_names=(ACTUATED,))
```

- `Wall.load` lit `simulator/climbing_sim/walls/N1.json` : la liste des prises (`Hold(col, row)`) et la prise de départ de chaque membre.
- `wall.start_sockets()` renvoie un tableau numpy (4, 3) : la position x, y, z en mètres du centre de chaque coupe de départ. `.tolist()` le convertit en listes Python simples, car la config doit pouvoir être copiée et envoyée à W&B. La conversion en tenseur GPU se fait au dernier moment dans `mdp.py`.
- `SceneEntityCfg` est un **sélecteur** : « l'entité `robot`, seulement ces articulations ». `ACTUATED` est l'expression régulière « tous les noms sauf `head` » : les 18 articulations commandées. Il sert pour `joint_pos`, `joint_vel` et `joint_limits`.

3.2 La scène

Champ	Rôle
`entities={"robot": climber_cfg(...)}`	le robot (modèle, servos BAM, pose accrochée), défini dans `robot.py` ; les fonctions le retrouvent par `env.scene["robot"]`
`spec_fn=wall_spec_fn(...)`	ajoute le sol, le mur et les prises (`scene.py`)
`terrain=None`	pas de grille : tous les robots au même endroit, chacun dans son propre monde
`num_envs=1`	écrasé par `--env.scene.num-envs`

Toutes les prises ont des collisions : les petites sphères de la coupe, où repose la boule. Celles de `mesh_holds` reçoivent **en plus** les maillages de leur corps (plaque, goussets), pour que la tige et le pied ne les traversent pas.

3.3 Les actions

Le réseau sort 18 nombres, un par servo (la tête n'est pas commandée par la politique). Chacun devient une consigne :

```
consigne = angle de la pose accrochée + 0,5 × action
```

Action nulle = le robot tient la pose de départ. `scale=0.5` limite l'amplitude à environ ±0,5 rad autour de la pose. Ensuite, **BAM** transforme la consigne en couple réaliste.

BAM (*Better Actuator Models*, Rhoban) simule les servos XL330 tels que mesurés au banc : couple limité, chute de tension sous charge, limite de courant 1,75 A, frottements dépendant de la charge, latence de 10 à 30 ms. Sans lui, la politique apprendrait des gestes impossibles pour le vrai robot.

3.4 Les observations

L'**actor** (le joueur) ne voit que ce que le vrai robot mesure, soit 60 nombres :

Observation	Contenu	Taille
`base_ang_vel`	vitesse de rotation du tronc (gyroscope)	3
`projected_gravity`	direction de la gravité vue du tronc	3
`joint_pos`	angle de chaque servo, relatif à la pose accrochée	18

Observation	Contenu	Taille
`joint_vel`	vitesse de chaque servo	18
`actions`	consigne du pas précédent (à cause de la latence des servos)	18

Chaque observation de l'actor reçoit un **bruit** (`noise`) et un **retard** de 0 à 3 pas (`delay`), comme les vrais capteurs. Ainsi la politique ne dépend pas d'informations parfaites qu'elle n'aura jamais sur le robot. Ce bruit n'est actif qu'à l'entraînement (`enable_corruption=not play`).

Le **critic** (le coach) voit les mêmes 60 nombres sans bruit, plus 19 de vérité terrain, soit 79 : `base_lin_vel` (vitesse de déplacement du tronc, 3), `ball_offsets` (vecteur boule → coupe cible en x, y, z pour les 4 boules, 4 × 3 = 12), `hooked` (chaque boule dans sa coupe cible à moins de 6 mm, 4). Il peut « tricher » parce qu'il ne sert qu'à l'entraînement : mieux il voit, mieux il juge l'actor. Dans la base, ses cibles sont les coupes de départ ; chaque tâche peut les remplacer.

3.5 Les événements

Mode	Quand	Ici
`startup`	une fois, à la création	`bam_init`
`reset`	au début de chaque épisode	`reset_scene_to_default` : retour à la pose accrochée
`interval`	toutes les X secondes	non utilisé (c'était les poussées du G1)

`bam_init` règle un détail de mémoire. Par défaut, mjlab garde **une seule copie** des réglages du modèle, partagée par tous les robots. Or BAM écrit des frottements **différents pour chaque robot** (selon l'effort du moment, et tirés au hasard à ±10 %). `bam_init` demande donc à mjlab une case par robot pour ces frottements. Une seule fois au démarrage, puisqu'il s'agit juste de réserver la place.

3.6 Les terminaisons

Nom	Condition	Type
`time_out`	temps écoulé	pas un échec (`time_out=True`)

Nom	Condition	Type
`fell`	tronc descendu de plus de 5 cm sous sa hauteur de départ (`start_z`)	échec
`nan`	valeurs invalides (NaN) : la simulation a explosé	sécurité

`time_out=True` indique à PPO que l'épisode s'arrête parce qu'on a coupé le chrono, pas parce que le robot a échoué. Sans cela, il verrait la fin du temps comme une punition.

3.7 Les pénalités communes

- `action_rate` (poids −0,01) : grands changements d'action d'un pas à l'autre, c'est-à-dire des saccades.
- `joint_limits` (poids −1) : articulations proches de leurs butées.

Un poids négatif est une pénalité.

3.8 La physique

Réglage	Valeur	Sens
`njmax` / `nconmax`	400 / 128	place réservée par robot pour les contraintes et les contacts ; trop petit = contacts ignorés (`nefc overflow`)
`timestep`	0,004	pas de 4 ms
`integrator`	implicitfast	méthode stable, celle du simulateur officiel
`cone`	pyramidal	approximation du frottement, idem
`iterations`	20	passes du solveur de contacts
`episode_length_s`	10	durée par défaut d'un épisode

Le bloc `viewer` règle seulement la caméra de `play` .

4. La tâche Hang : le motif « base + règles »

Hang demande au robot de rester suspendu aux 4 prises de départ. Toute tâche suit le même schéma : prendre la base, puis ajouter ses règles dans les dictionnaires de la config.

```
def hang_env_cfg(play=False):
  cfg = _base_cfg(play)                                  # 1. la base
  sockets = [...]                                        # 2. les 4 coupes de
départ
  cfg.rewards["near_sockets"] = RewardTermCfg(...)   # 3. les règles propres
à la tâche
  cfg.rewards["hooked"] = RewardTermCfg(...)
  if play:
    cfg.episode_length_s = 1e9                           # 4. en play : pas de
limite de temps
  return cfg
```

Les deux récompenses se complètent :

- **`near_sockets`** (poids 1) est progressive : chaque boule reçoit une note de 0 à 1 selon sa distance au centre de sa coupe (courbe en cloche, σ = 1 cm : 1 au centre, ~0,37 à 1 cm, presque 0 à 2 cm), puis on fait la moyenne des 4. Elle **guide**.
- **`hooked`** (poids 1) est tout ou rien par boule : 1 si elle est à moins de 6 mm du centre de sa coupe, sinon 0, puis moyenne des 4 (0 ; 0,25 ; 0,5 ; 0,75 ou 1). Elle **récompense le résultat**.

Une récompense progressive pour guider plus un bonus pour le but : c'est le schéma classique contre le problème des récompenses rares (section 6). Avec seulement le bonus, le robot n'aurait aucune piste tant qu'il ne l'aurait pas trouvé par hasard.

La ligne `1e9`. Un épisode s'arrête quand le temps est écoulé (10 s à l'entraînement) ou quand le robot échoue (`fell`). En `play` seulement, la durée passe à un milliard de secondes : le robot n'est plus remis à zéro toutes les 10 secondes, il reste suspendu aussi longtemps qu'il tient. S'il tombe, l'épisode s'arrête quand même.

5. La tâche LeftFoot, ligne par ligne

LeftFoot fait passer le pied gauche de sa prise de départ (21, 2) à la prise en haut à gauche (18, 7), les deux mains et le pied droit restant accrochés. La prise juste au-dessus (21, 7) est hors de portée, comme l'équipe l'avait constaté à la main sur le vrai robot.

5.1 Choisir la cible

```
wall = Wall.load(WALL)
start = wall.start_sockets()                    # positions (x, y, z) des 4
coupes de départ
foot = wall.start["left_foot"]                  # Hold(21, 2) : la prise, sur
la grille
target_hold = Hold(foot.col - 3, foot.row + 5)  # Hold(18, 7)
if target_hold not in wall.holds:
  raise ValueError(...)
```

`start` et `foot` ne font pas doublon. `start` donne des **positions en mètres**, `foot` une **prise sur la grille** (colonne, rangée). « 3 colonnes à gauche, 5 rangées plus haut » ne se calcule qu'avec la grille ; `target_hold.socket` reconvertit ensuite en position.

La vérification arrête le programme avec un message clair si la prise n'existe pas, au lieu d'entraîner le robot vers une prise fantôme.

5.2 La base, avec les bons maillages

```
cfg = _base_cfg(play, mesh_holds=tuple(h for h in wall.holds if h.row in
(foot.row, target_hold.row)))
```

Les prises des rangées 2 et 7, là où passe le pied, reçoivent les maillages de leur corps. Coût mesuré : environ −19 % de vitesse.

5.3 Les cibles

```
goal = start.copy()                    # copie des 4 coupes de départ
goal[LEFT_FOOT] = target_hold.socket   # la nouvelle coupe du pied gauche
goal = [g.tolist() for g in goal]
```

`goal` dit où chaque boule doit être : 3 coupes de départ et la nouvelle coupe du pied gauche. Sans `.copy()`, `goal` et `start` seraient le même tableau en mémoire. Les deux lignes suivantes donnent ces cibles au critic (`ball_offsets`, `hooked`), sinon il jugerait le pied par rapport à son ancienne prise.

5.4 Les récompenses

Régle	Fonction	Poids	Rôle
`others_hooked`	`balls_near_sockets`, membres 0, 1, 3, σ = 1 cm	1	maintien : note de 0 à 1 par boule, moyenne des 3

Régle	Fonction	Poids	Rôle
`foot_reach`	`limb_reach`, membre 2, σ = 4 cm	2	guidage : note de 0 à 1 selon la distance du pied à sa cible
`foot_hooked`	`limb_hooked`, membre 2	5	but : pied à moins de 6 mm du centre de la nouvelle coupe

Les deux σ diffèrent : 4 cm pour le pied, qui part à 7,5 cm de sa cible et doit la « sentir » de loin ; 1 cm pour les autres, qui doivent rester au fond de leur coupe. Les récompenses s'additionnent : pied dans la prise = 1 + 2 + 5, soit 8 au maximum par seconde.

5.5 La terminaison propre à la tâche

`limb_lost` arrête l'épisode si une main ou le pied droit s'éloigne de plus de 3 cm de sa prise. Un épisode coupé, ce sont tous les points futurs perdus : le robot apprend à l'éviter sans qu'il faille écrire de récompense négative.

5.6 Durée

8 s par épisode à l'entraînement, largement assez pour un mouvement de pied ; pas de limite de temps en `play`.

6. Le vocabulaire des récompenses

Dans la littérature (en anglais), on décrit une récompense selon deux questions indépendantes : **comment** le signal est donné, et **à quoi** il sert.

Terme	Sens	Ici
Dense reward, *shaped reward* (procédé : *reward shaping*)	donnée à chaque pas, varie en continu, indique la direction	`near_sockets`, `foot_reach`, `others_hooked`
Sparse reward, *binary reward*, *success bonus*	donnée seulement quand le but est atteint	`hooked`, `foot_hooked`
Tracking reward	dense, pour rester proche d'une valeur cible	`others_hooked` (les appuis restent au fond de leur coupe)

Terme	Sens	Ici
Penalties, *regularization terms*	ne disent pas quoi faire, rendent le comportement propre et sûr	`action_rate`, `joint_limits`

`others_hooked` et `foot_reach` utilisent la même fonction, une cloche $\exp(-d^2/\sigma^2)$ appelée **noyau gaussien**. Seuls le σ et le rôle changent. Le schéma guide (dense) + bonus (sparse) est la réponse classique au *sparse reward problem* : un bonus rare seul est presque impossible à découvrir par exploration.

Contraintes douces et dures.

- Une **contrainte douce** peut être enfreinte, mais coûte des points. `others_hooked` en est une : le robot peut laisser glisser une main de quelques millimètres si ça l'aide à attraper la prise du pied. Il apprend à faire le compromis.
- Une **contrainte dure** met fin à l'épisode. `limb_lost` en est une : au-delà de 3 cm, plus de compromis possible.

Les deux travaillent ensemble sur les mêmes membres : de 0 à 3 cm, `others_hooked` règle finement ; au-delà, `limb_lost` interdit.

L'échelle est par seconde. mjlab multiplie chaque récompense par la durée d'un pas (0,02 s), pour que les totaux ne dépendent pas de la fréquence de décision. Note × poids est donc un taux **par seconde**. Sur W&B, les courbes `Episode_Reward/...` sont des moyennes par seconde sur l'épisode et sur tous les robots : `foot_hooked` plafonne à 5, et une valeur de 2,5 veut dire que le pied est dans sa coupe environ la moitié du temps, en moyenne. `Train/mean_reward`, lui, est le total sur l'épisode.

7. Les réglages PPO

`ppo_cfg` reprend les réglages de `mjlab_microban` (la tâche de marche de Rhoban). On n'y touche presque jamais ; voici ce qui compte.

Réglage	Valeur	Signification concrète
`hidden_dims`	512, 256, 128	taille du réseau (petit MLP, ~220 000 paramètres)
`init_std`	1,0	exploration au départ : bruit de 1 sur les actions, donc ±0,5 rad avec `scale=0.5`
`obs_normalization`	oui	met les observations à la même échelle (rad, rad/s)

Réglage	Valeur	Signification concrète
`num_steps_per_env`	24	pas joués par robot à chaque itération (0,48 s) ; avec 6144 robots, 147 456 expériences par itération
`num_learning_epochs` / `num_mini_batches`	5 / 4	PPO repasse 5 fois sur le lot, découpé en 4 morceaux
`clip_param`	0,2	garde-fou de PPO : pas plus de ±20 % de changement à la fois sur chaque action
`gamma`	0,99	horizon d'anticipation d'environ 100 pas, soit ~2 s
`entropy_coef`	0,01	bonus pour continuer à explorer
`save_interval`	100	un checkpoint toutes les 100 itérations (`model_100.pt`, `model_200.pt`…)

`max_iterations` est donné à l'enregistrement de la tâche, dans `tasks/__init__.py`. Pour reprendre un entraînement interrompu : `--agent.resume True --agent.load-run <dossier> --agent.load-checkpoint model_N.pt`.

8. Le film d'un pas de 20 ms

Pour chacun des robots, en même temps sur le GPU :

1. **Observations** : l'actor reçoit ses 60 nombres, bruités et retardés.
2. **Décision** : le réseau sort 18 actions, plus le bruit d'exploration.
3. **Actions** : consigne = pose accrochée + 0,5 × action.
4. **Physique** : 5 sous-pas de 4 ms ; à chaque sous-pas, BAM calcule le couple des servos, puis MuJoCo Warp avance le monde.
5. **Terminaisons** : tombé ? membre perdu ? temps écoulé ?
6. **Récompenses** : somme des règles × poids × 0,02 s.
7. **Reset** : les robots dont l'épisode est fini repartent de la pose accrochée.

Tous les 24 pas, PPO apprend à partir des expériences récoltées (avantage = points obtenus − points prévus par le critic ; montée de gradient sur log P(action) × avantage, clippée à ±20 %), sauvegarde si besoin, et on recommence avec la nouvelle version de l'actor.

9. Pour vérifier que tu as compris

1. Si je mets `scale=1.0` dans les actions, qu'est-ce qui change au début de l'entraînement ? L'amplitude double (±1 rad autour de la pose). Le bruit d'exploration initial passe aussi par `scale` : le robot gigote deux fois plus fort, touche plus souvent ses butées et lâche plus souvent ses prises. L'apprentissage démarre plus difficilement.

2. Pourquoi `others_hooked` utilise σ = 1 cm, mais `foot_reach` σ = 4 cm ? Le pied part à ~7,5 cm de sa cible : avec 1 cm, la récompense serait quasi nulle sur tout le trajet et ne guiderait rien. Les trois appuis, eux, doivent rester au fond de leur coupe : un σ serré punit le moindre glissement.

3. Que se passerait-il si on retirait `time_out=True` ? La fin du chrono serait traitée comme une vraie fin, sans points possibles ensuite. Le critic verrait la valeur s'effondrer en fin d'épisode sans que rien dans les observations l'explique (le robot ne voit pas l'horloge) : estimations bruitées et comportements bizarres en fin d'épisode.

4. Le robot garde le pied immobile pour ne jamais perdre ses appuis. Que changer ? Faire que bouger rapporte plus : augmenter le poids de `foot_reach`, ou baisser celui de `others_hooked`, ou assouplir `limb_lost` (`max_dist` plus grand). Un seul changement à la fois.

5. Pourquoi fusionner les récompenses des deux pieds en une moyenne ne change rien à l'apprentissage ? 4 × (gauche + droit) / 2 = 2 × gauche + 2 × droit : même récompense, à condition de doubler le poids. Mais on perd une courbe par pied sur W&B. Une vraie différence viendrait d'un « et » : récompenser seulement les deux pieds accrochés ensemble.

6. Exercice : écris la tâche du pied droit, de (26, 2) vers (29, 7). Copier `left_foot_env_cfg` et changer : `wall.start["right_foot"]` ; `RIGHT_FOOT = LIMBS.index("right_foot")` (3) et les autres membres `(0, 1, 2)` ; la cible `Hold(foot.col + 3, foot.row + 5)` ; chaque clé de récompense unique. Puis l'enregistrer dans `tasks/__init__.py` et lancer `uv run play <tâche> --agent zero` pour vérifier qu'elle se construit.